

Parameter Estimation

Marijn

January 25, 2026

1 Introduction

We have our dynamical system

$$\dot{x} = f(x, u), \quad (1)$$

but now consider it to be dependent on some set of parameters θ :

$$\dot{x} = f(x, u; \theta). \quad (2)$$

For example, if we take the van der Pol oscillator with a control term

$$\begin{aligned} \dot{x}_1(t) &= x_2(t) \\ \dot{x}_2(t) &= -x_1(t) + \mu(1 - x_1(t)^2)x_2(t) + u(t), \end{aligned} \quad (3)$$

then, in this example, we can take the damping coefficient μ as our parameter, making the set of parameters: $\theta = \mu$.

Now, given our system (2) with a fixed parameter θ^* , we are interested in estimating this parameter from trajectory data. Roughly, this can be divided into two steps: 1) Learning a surrogate model of the parametric flow function using data generated with various parameter values, now with the parameter incorporated as input in the flow function. 2) Estimating the parameter using a surrogate model by comparing its output with some estimated parameter $\hat{\theta}$ with data generated with true parameter θ^* .

2 Flow function with varying parameters

With parameter $\theta \in \Theta$, the *flow*, describing the evolution of state-variables, can now be given as the following mapping:

$$\varphi : \mathcal{T} \times \mathcal{X} \times \Theta \times \mathbb{U} \rightarrow \mathcal{X} \quad (4)$$

2.1 Proposed architecture

While other possibilities may be considered, we first choose to include the parameter in the encoder, as addition to the initial condition x_0 , as this ensures that universal approximation is preserved. A block diagram of this architecture is shown in Figure 1.

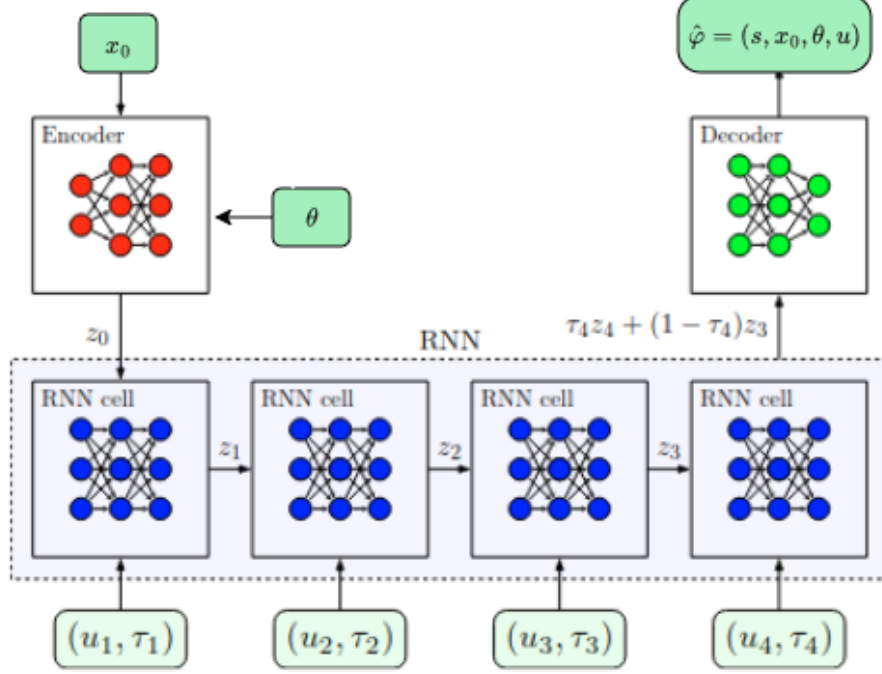


Figure 1: Model for the approximated flow $\hat{\varphi}$, now with a parameter in the encoder

2.2 Learning the flow

To learn the flow function, we collect trajectory data generated with (2). In practice, this data consists of discrete-time samples from N different trajectories:

$$\xi_k^i = \varphi(t_k^i, x^i, \theta^i, u^i), \quad k = 1, \dots, K, \quad i = 1, \dots, N, \quad (5)$$

where K is the number of samples of each trajectory, $t_k^i \in [0, T]$ is an increasing sequence of time samples, and x^i , u^i and θ^i are samples i.i.d from probability distributions P_x , P_u and P_θ respectively. We can then define the following loss function:

$$\ell(\hat{\varphi}) := \frac{1}{N} \sum_{i=1}^N \frac{1}{K} \sum_{k=1}^K \|\xi_k^i - \hat{\varphi}(t_k^i, x^i, \theta^i, u^i)\|^2, \quad (6)$$

and train a neural network to minimise (6).

2.3 Experimental evaluation

We consider the van der pol oscillator as described in (3), with the damping coefficient uniformly distributed between 0 and 2, i.e. $\theta \sim U(0, 2)$. Other data settings be seen in Appendix A. The test loss of models trained on different amounts of training trajectories N can be seen in Table 1. Figure 2 shows the validation losses per epoch. The associated training (and model) settings can be seen in Appendix B. There is a noticeable difference between the performance of the different models, implying a significant amount of data is required to get an acceptable loss. This can most likely be attributed to the introduction of

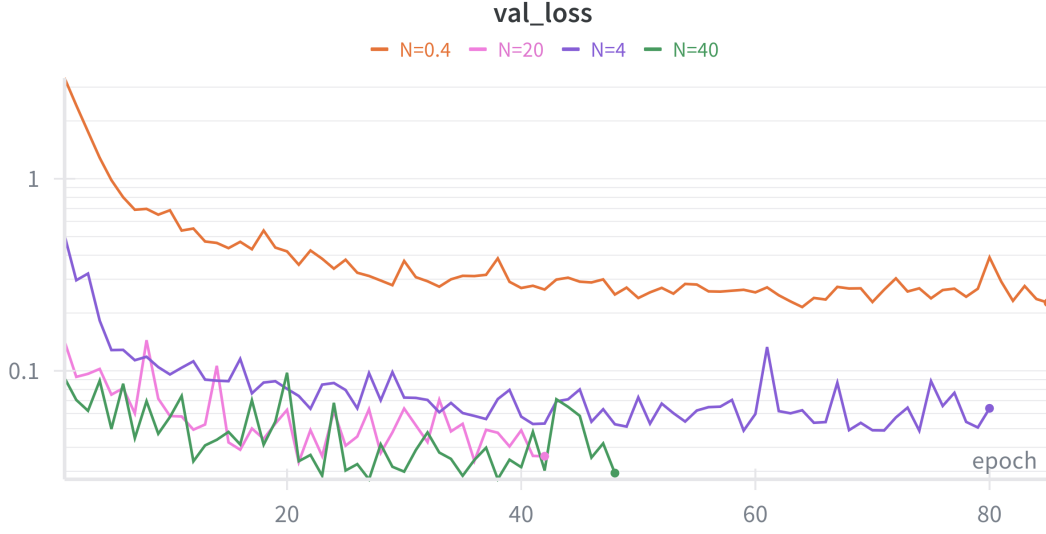


Figure 2: Validation loss per epoch. Logarithmic Y-scale.

an additional unique variable per trajectory (alongside the initial state x^i and control input u^i) resulting in a larger set of unique trajectories.

Table 1: Loss for models trained with trajectories N

Trajectories N [10^3]	Loss
0.4	0.2598
4	0.0575
20	0.0359
40	0.0284

We also compare the best model in Table 1 with a baseline model trained on $N = 3000$ trajectories generated with a fixed parameter of value $\theta = 1.0$, the results on trajectories with this parameter value are shown in Table 2. The model with varying parameters reaches the comparable performance as the baseline model, meaning that $N = 40 \times 10^3$ is an ample amount of trajectories, and there is no use in adding more trajectories. Furthermore, if we assume the baseline flow function demonstrates good performance, then the comparable results suggest that for the parametric flow function, at least for $\theta = 1.0$, is successfully incorporating the parameter.

Table 2: Comparison baseline ($\theta = 1.0$) and model with varying parameters ($\theta \sim U(0, 2)$)

Model	Loss
Baseline	0.02163
Varying parameter	0.02166

3 Estimating the parameter

Given our dynamics (2), we are now interested in estimating the parameter θ from data. If we have trajectory data with some fixed parameter θ ,

$$y_{\theta}^i(t_k) = \varphi(t_k^i, x^i, u^i; \theta), \quad k = 1, \dots, K, \quad i = 1, \dots, N, \quad (7)$$

then, we find the estimated parameter $\hat{\theta}$ by minimizing the residual between the observed data and our model approximation, i.e.

$$\ell(\hat{\theta}) := \frac{1}{N} \sum_{i=1}^N \frac{1}{K} \sum_{k=1}^K \|y_{\theta}^i(t_k) - \hat{y}_{\hat{\theta}}^i(t_k)\|^2. \quad (8)$$

The model approximations $\hat{y}_{\hat{\theta}}^i$ consist of our flow function $\hat{\varphi}$ and the numerical solver *diffraX* [1]. Both model approximations are implemented using the JAX library [2], offering computational advantages in the forms of efficient automatic differentiation, parallel computations, and runtime advantages through Just-In-Time (JIT) compilation. As we have access to the gradients of our output with respect to the parameter, i.e. $\frac{\partial \hat{y}}{\partial \theta}$, we can minimize our loss (8) using gradient-based optimization techniques, such as gradient descent or Adam.

3.1 Results

We explore both the accuracy and computation time of the flow function and compare it to the numerical solver *diffraX* [1]. We collect $N = 100$ trajectories with (3) (generated using the data settings as described in Appendix A), using a value of $\mu = \theta^* = 1.0$. For *diffraX*, Dopri5 was used as integrator method, with an initial time step size of dt_0 . This initial time step size was adjusted for the different starting parameters to enforce stability. For the minimisation algorithms, we use standard gradient descent with learning rate of 0.02 and Broyden-Fletcher-Goldfarb-Shanno (BFGS), both implemented using the *optimistix* library [3].

The results for both the parametric flow function *flumen*, and the numerical solver *diffraX*, where $\hat{\theta}_0$ denotes the initial parameter guess, can be seen in Table 3. A visualization of the estimated parameter per minimization step, starting with $\hat{\theta}_0 = 0.5$, can be seen in Figure 3. For those same respective steps, the loss as defined in (8) can be seen in Figure 4. The results for the more advanced algorithm, BFGS, can be seen in Table 4. To give some extra context, a visualization of the loss between the data and the respective surrogate models as a function of the parameter, as given in (8), can be seen in Figure 5.

A Data settings

The data settings can be seen in Table 5.

B Training settings

The training settings can be seen in Table 6.

Table 3: Performance comparison between `flumen` and `diffraX` for different estimation using gradient descent for different starting points $\hat{\theta}_0$. True parameter $\theta = 1.0$. Computation time is averaged over 5 runs. $^\dagger dt_0 = 0.1$, $^* dt_0 = 0.01$.

$\hat{\theta}_0$	Finds True Parameter?		Computation Time (s)	
	<code>flumen</code>	<code>diffraX</code>	<code>flumen</code>	<code>diffraX</code>
-0.5	✗	✗	-	-
0.0	✓	✗	0.80	6.33*
0.5	✓	✓	0.82	0.69 [†]
1.5	✓	✓	0.92	0.80 [†]
2.0	✓	✓	1.66	3.54*
2.5	✓	✓	3.32	6.22*
3.0	✓	✗	5.43	-
3.5	✓	✗	10.43	-
4.0	✗	✗	-	-

References

- [1] Patrick Kidger. DiffraX, 2024. URL: <https://docs.kidger.site/diffrax/>.
- [2] Jax. URL: <https://docs.jax.dev/en/latest/about.html>.
- [3] Patrick Kidger. Optimistix, 2024. URL: <https://docs.kidger.site/optimistix/>.

Table 4: Performance comparison between **flumen** and **diffrax** for different estimation using BFGS for different starting points $\hat{\theta}_0$. True parameter $\theta = 1.0$. Computation time is averaged over 5 runs. $^\dagger dt_0 = 0.1$, $^* dt_0 = 0.01$.

$\hat{\theta}_0$	Finds True Parameter?		Computation Time (s)	
	flumen	diffrax	flumen	diffrax
-0.5	✗	✗	-	-
0.0	✓	✓	1.04	2.16 [†]
0.5	✓	✓	0.57	0.51 [†]
1.5	✓	✓	0.57	0.46 [†]
2.0	✓	✓	0.49	1.07 [*]
2.5	✓	✓	1.24	1.91 [*]
3.0	✓	✓	0.80	1.29 [*]
3.5	✓	✓	0.82	1.47 [*]
4.5	✓	✓	0.80	1.72 [*]
5.0	✗	✓	-	1.72 [*]
5.5	✗	✗	-	-

Table 5: Data Settings

Parameter	Value
T	10
P_x	$U(-4, 4)$
P_u	$N(0, 1)$
u period	1
Control delta	1
K	50

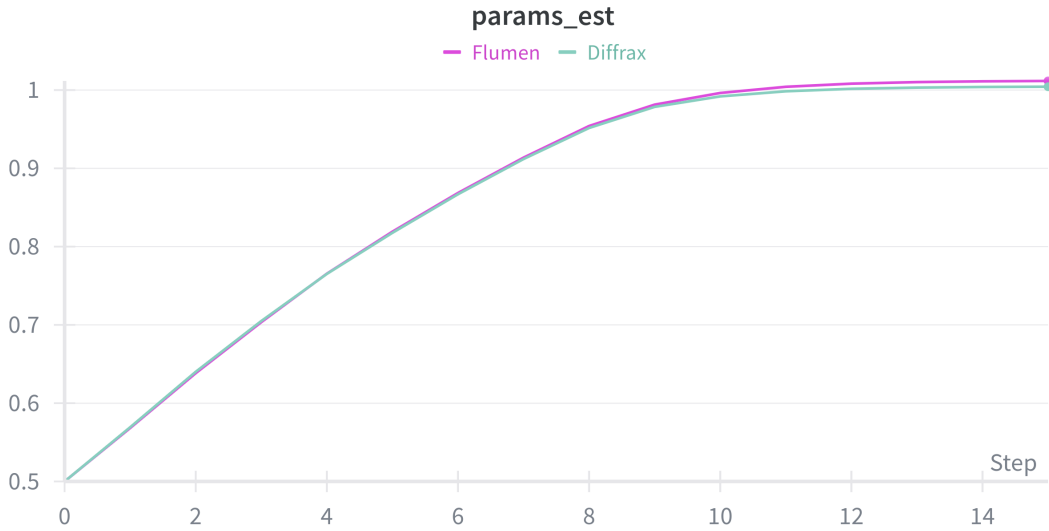


Figure 3: Estimated parameter per gradient descent step.



Figure 4: State loss per gradient descent step.

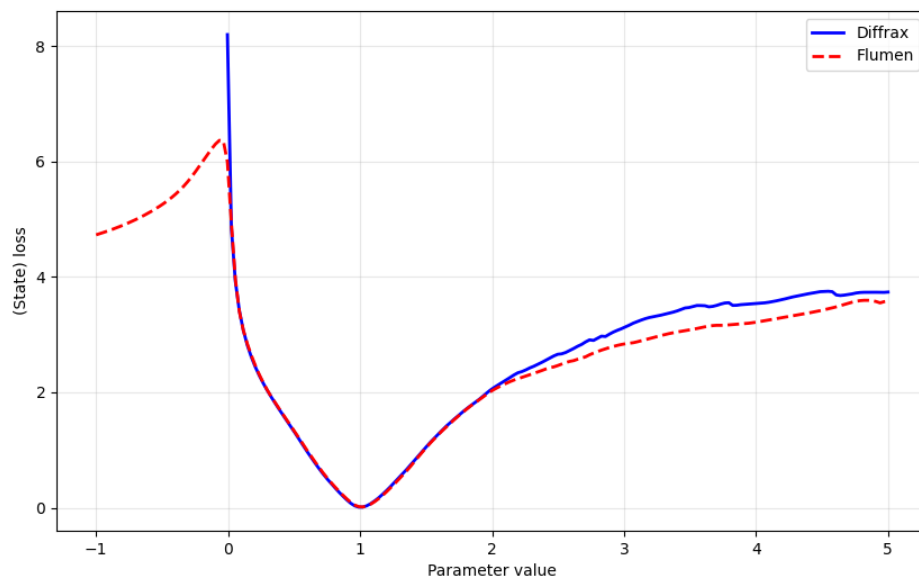


Figure 5: State loss per parameter ($\theta^* = 1.0$).

Table 6: Training Settings

Parameter	Value
Batch size	128
learning rate	5×10^{-4}
Hidden size	32
Encoder size	[128, 128]
Decoder size	[128, 128]